SPENDWISE / IMPLEMENTATION GUIDE

Supabase & Google OAuth Setup

Apply the new database migrations, configure authentication, deploy secure account deletion, and verify the integration before using real financial data.

What you will set up

- Six ordered SQL migrations: core schema, profile names, finance features, notifications and Realtime, recurring income and loan links, and private receipts.
- The public Supabase URL/key in the app environment (never the service-role key).
- Supabase Auth redirect URLs, confirmation/reset flows, and Google OAuth credentials.
- The delete-account Edge Function, which uses a server-side admin key and verifies the signed-in user.
- A basic web smoke test and an optional live two-account RLS test.

Before you begin

You need access to the SpendWise project files, a Supabase project with database access, and a website origin (localhost is fine for initial testing). Google sign-in additionally needs a Google Cloud project. The Android app also needs its custom scheme redirect to be allow-listed.

Important: This repository is not connected to your Supabase project. You must select the correct project, apply migrations, enter project-specific values, and enable providers yourself. Never place a service-role key in the browser app, .env.local, Git, screenshots, or this PDF.

Recommended order

1. Back up any existing test data. 2. Apply migrations. 3. Configure public app values. 4. Configure Auth URLs and Google. 5. Deploy the deletion function. 6. Run checks and test with disposable accounts.

1. Apply the database migrations

Use the Supabase Dashboard SQL Editor or the Supabase CLI. Run the migrations in order; later files depend on earlier tables.

Dashboard method

In Supabase, open the intended project and confirm its name/ref in the project switcher. Open **SQL Editor**, then run each migration file in a separate query, in this order:

- `supabase/migrations/20261030000000_initial.sql - profiles, transactions, budgets, core RLS`
- `supabase/migrations/20261030000001_profile_names.sql - profile name trigger and backfill`
- `supabase/migrations/20261104000000_finance_features.sql - loans, recurring bills, shared expenses`
- `supabase/migrations/20261105000000_notifications_realtime.sql - notifications, budget alerts, Realtime publication`
- `supabase/migrations/20261105000001_recurring_income_and_loan_links.sql - recurring item type and loan-linked transactions`
- `supabase/migrations/20261105000002_receipt_attachments.sql - private receipts bucket and storage RLS`

For each file: open it from the SpendWise workspace, copy the complete SQL, paste it into a new SQL Editor query, review the target project, and click **Run**. Wait for success before running the next file. Do not edit the SQL to add project keys.

The receipt migration expects Supabase Storage tables and schema to be available (they are part of Supabase). It creates a private bucket with a 5 MB limit and owner-folder policies. The Realtime migration adds tables to the publication only when that publication exists.

CLI method (alternative)

From PowerShell in the repository root, install/use the Supabase CLI, authenticate, link the exact project, inspect migration status, then push:

```
npx supabase@latest login
npx supabase@latest link --project-ref YOUR_PROJECT_REF
npx supabase@latest migration list
npx supabase@latest db push
```

Replace `YOUR_PROJECT_REF` with the ref shown in the project URL (the subdomain before `.supabase.co`). The CLI may ask for the database password. Do not paste it into source files or commit it. Check the CLI output and the migration history before rerunning anything.

Do not run both methods blindly. If migrations have already been applied, use migration history to determine the next unapplied file. These SQL changes are not a substitute for a backup or a reviewed rollback plan.

EXISTING PROJECT? APPLY ONLY WHAT IS NEW

If the original three migrations (initial, profile names, and finance features) are already applied, do not run them again. Apply only the final three 20261105000000 through 20261105000002 migration files. If the earlier migrations were applied manually in SQL Editor, keep using SQL Editor for these new files. Use CLI db push only when remote migration history accurately records the earlier versions as applied.

2. Configure the SpendWise app

The website uses only the project URL and publishable/anon key. The server-side deletion function uses a separate privileged key that must never reach the client.

Get the public values

In the Supabase Dashboard, open **Project Settings / API** (or **Connect** / **API Keys**, depending on the dashboard layout). Copy the project URL and the publishable key (legacy anon key is also supported). The URL looks like `https://PROJECT_REF.supabase.co`.

Create the local environment file

```
NEXT_PUBLIC_SUPABASE_URL=https://YOUR_PROJECT_REF.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=YOUR_PUBLISHABLE_OR_ANON_KEY
```

Save these lines as `.env.local` in the project root (same folder as package.json). Do not add quotes unless your value requires them. This file is gitignored. Restart the dev server after changing it:

```
npm run dev
```

Never use a secret key or service_role key for NEXT_PUBLIC variables. Next.js exposes NEXT_PUBLIC_* values to the browser. The publishable/anon key is intended for clients and relies on Auth plus RLS for access control.

Production website

Deploy the normal Next.js server build, not the Android static export folder. In the hosting provider, set the same two public variables in the production build/runtime environment, then redeploy. The configured website must be served from HTTPS for real users.

3. Configure Supabase Auth URLs

Redirect URLs must match the browser or Android callback that the app requests. Add only the origins and paths you actually use.

URL Configuration

In Supabase Dashboard, open **Authentication / URL Configuration**. Set **Site URL** to your deployed website origin, for example `https://spendwise.example.com`. For local testing it can be `http://localhost:3000`. Add these Redirect URLs, replacing the example domain:

- `http://localhost:3000/auth/confirm`
- `http://localhost:3000/auth/callback`
- `https://YOUR-DOMAIN/auth/confirm`
- `https://YOUR-DOMAIN/auth/callback`
- `spendwise://auth/confirm`
- `spendwise://auth/callback`

If using preview deployments, add their exact callback URLs too. The reset flow returns to `/auth/callback?next=/auth/reset-password`; the Android reset callback includes the same next path on the `spendwise://` scheme. Keep redirect allow-list entries scoped to your app rather than allowing arbitrary URLs.

Email templates and delivery

The app handles confirmation links at `/auth/confirm` and PKCE/recovery callbacks at `/auth/callback`. If you want confirmation links to work across devices, update the **Confirm signup** template to use a token-hash link:

```
{{ .SiteURL }}/auth/confirm?token_hash={{ .TokenHash }}&type=email
```

Keep the reset-password email action pointed at the redirect URL requested by the app. Configure a production SMTP provider in Supabase for dependable delivery; built-in email sending is rate-limited. Test signup confirmation, password recovery, and email change with a real test mailbox.

For Android links, the installed app must recognize the spendwise scheme. The repository already declares spendwise://auth in AndroidManifest.xml and routes confirm/callback paths. Rebuild and install the app after changing native configuration.

4. Set up Google OAuth

Google credentials are entered in the Supabase provider settings. The client secret stays in Supabase; it is never added to SpendWise source or .env.local.

Create the Google OAuth client

- Open Google Cloud Console and select/create a project. Configure the OAuth consent screen (branding, audience, contact email, and required scopes). For External / Testing mode, add the Google accounts that will test sign-in.
- Create an OAuth Client ID of type Web application. Give it a recognizable name such as SpendWise Supabase Auth.
- In Supabase Authentication / Providers / Google, copy the callback URL shown by Supabase. It normally has this form:
- Add that exact Supabase callback URL to Google Authorized redirect URIs. Add your website origin under Authorized JavaScript origins if Google requests it. Save the client.
- Copy the Google Client ID and Client Secret into the Supabase Google provider fields and enable the provider. Save changes. Do not put either value into the web app.

```
https://YOUR_PROJECT_REF.supabase.co/auth/v1/callback
```

Use the exact callback URI displayed in your Supabase project; do not guess the project ref or add the SpendWise custom scheme as Google?s callback. Google returns to Supabase first. Supabase then redirects to the SpendWise URL requested during sign-in.

Test the provider

Open `/login` or `/signup`, choose **Continue with Google**, approve with a test Google account, and confirm the app lands on the dashboard. For Android, test on an installed build and confirm the browser returns to SpendWise through `spendwise://auth/callback`. If Google reports a redirect_uri_mismatch, compare the exact URI in the error with the Authorized redirect URI in Google Cloud.

Google?s OAuth consent screen may remain in Testing until you publish it. Add test accounts there. A production release may require Google verification depending on audience, scopes, and distribution.

5. Deploy account deletion

The profile page calls a Supabase Edge Function. The function validates the caller token, deletes that user?s private receipt files, then deletes the Auth user so owner-linked rows cascade.

Deploy from the repository root

```
npx supabase@latest login
npx supabase@latest link --project-ref YOUR_PROJECT_REF
npx supabase@latest functions deploy delete-account --project-ref YOUR_PROJECT_REF
```

Confirm the deployment targets the same project used by the app. The function expects Supabase?s server-side environment values `SUPABASE_URL`, `SUPABASE_ANON_KEY`, and `SUPABASE_SERVICE_ROLE_KEY`. Supabase normally injects these for Edge Functions; check the function logs/settings if the endpoint reports that deletion is not configured.

If you must add or rotate the service-role value, do it only in Supabase Edge Function secrets (or through a secure secrets workflow). Never expose it in the app, NEXT_PUBLIC_ variables, Git, chat, logs, or command history. The deployed function must require a valid caller JWT; do not deploy it with JWT verification disabled.

Test deletion safely

Use a disposable test account. Attach a test receipt, confirm the account can view it, then use Profile / Account security and type DELETE. Confirm the user is signed out, the Auth user is removed, and the receipt object is gone. Check function logs for errors. Do not test irreversible deletion against an account containing real data.

6. Verify setup and troubleshoot

Complete the checks before enabling real data or distributing an APK.

Local app checks

```
npm ci
npm run typecheck
npm run lint
npm test
npm run build
```

- Create a test account and verify the confirmation email.
- Sign out, request a password reset, open the link, and set a new password.
- Request an email change and confirm it using the new address.
- Sign in with Google on web; test Android deep-link return on an installed app.
- Create a budget, add expenses until it is reached, and confirm the in-app alert appears.
- Open the same account in two tabs, change a cloud record, and confirm the second tab refreshes.
- Upload and open a small receipt; verify it is private and accessible only to its owner.
- Create a recurring income and record it; confirm an income transaction appears. Record a loan payment and confirm the linked transaction and balance update.

Optional live RLS check

Create two disposable, confirmed Supabase users with no valuable data. In PowerShell, set environment variables for the project?s public values and both test logins in the current terminal only:

```
$env:NEXT_PUBLIC_SUPABASE_URL="https://YOUR_PROJECT_REF.supabase.co"
$env:NEXT_PUBLIC_SUPABASE_ANON_KEY="YOUR_PUBLISHABLE_OR_ANON_KEY"
$env:RLS_USER_A_EMAIL="test-a@example.com"
$env:RLS_USER_A_PASSWORD="TEST_PASSWORD_A"
$env:RLS_USER_B_EMAIL="test-b@example.com"
$env:RLS_USER_B_PASSWORD="TEST_PASSWORD_B"
npm run test:rls:live
```

The script creates short-lived test rows, checks cross-account select/insert/update/delete isolation and anonymous reads, then attempts cleanup. Use only disposable test accounts and verify the test output. It does not use a service-role key. Clear the terminal variables when done:

```
Remove-Item Env:NEXT_PUBLIC_SUPABASE_URL,Env:NEXT_PUBLIC_SUPABASE_ANON_KEY,Env:RLS_USER_A_EMAIL,Env
:RLS_USER_A_PASSWORD,Env:RLS_USER_B_EMAIL,Env:RLS_USER_B_PASSWORD
```

Common failures

Migration already applied	Check Supabase migration history. Do not blindly rerun non-idempotent ALTER/CREATE statements.
Invalid login / Auth redirect blocked	Verify the redirect URL exactly matches the Auth allow list, including scheme, host, and path.
Google redirect_uri_mismatch	Use the callback URL displayed in Supabase as Google?s Authorized redirect URI.
Password reset link returns to website, not app	For web use /auth/callback; for Android allow spendwise://auth/callback and test from an installed app.
Function returns 503	Check deployment target and server-side Edge Function env/secrets; never solve this by putting service_role in the client.
RLS test says credentials missing	Set all six documented environment variables in the same PowerShell session before running the npm script.
Realtime or receipt feature unavailable	Confirm all migrations succeeded, the user is signed in, Realtime is enabled, and the receipt bucket is private with policies.

Production readiness: migrations applied to the intended project; RLS verified with two accounts; provider and SMTP settings tested; delete function deployed and exercised only with a disposable user; production hosting secrets configured; Android deep links checked on a device.